Missing Values, Outliers & Encoding

Contents

1. Why Do We Need Data Preprocessing?

Real-world data is almost never clean. It has blank cells, strange values, and words where the computer expects numbers. A machine-learning model is only as good as the data we feed it. This is the famous rule:

"Garbage in, garbage out."

Data preprocessing is the step where we clean and prepare raw data so a model can learn from it properly. In this module we focus on three of the most common cleaning tasks:

1. **Missing values** — cells that are empty (no data was recorded).
2. **Outliers** — values that are far away from the rest and may be errors.
3. **Encoding** — converting text categories (like "Male", "IT") into numbers.

The usual order

A safe order of operations is: **(1)** look at the data → **(2)** fix missing values → **(3)** handle outliers → **(4)** encode categories. We follow this exact order in the practical section at the end.

2. Part 1 — Handling Missing Values

2.1 What Are Missing Values?

A **missing value** is simply a cell with no data in it. In pandas, missing values usually show up as `NaN` ("Not a Number") or `None`. For example, a survey form where a person skipped the "Age" question creates a missing value.

Definition: Missing Value

A data point that was expected but is *not present* in the dataset. It tells us "we do not know this value," which is different from a value of zero.

2.2 Why Does Data Go Missing? (Causes)

Understanding *why* a value is missing helps us choose the right fix. There are two ways to think about causes: practical reasons, and the statistical "mechanism."

2.2.1 Practical, everyday causes

- **Human error:** someone forgot to type the value or made a mistake.
- **Equipment / sensor failure:** a device stopped recording for a while.
- **Survey non-response:** people skip questions they find private (e.g. salary, age).
- **Data merging:** combining two datasets leaves gaps where records do not match.
- **Data corruption:** files get damaged during transfer or storage.
- **Not applicable:** the question simply does not apply (e.g. "spouse name" for a single person).

2.2.2 The three statistical mechanisms

Statisticians group missingness into three types. The names sound similar, so read carefully.

Definition: MCAR — Missing Completely At Random

The fact that a value is missing has *nothing* to do with any data, seen or unseen. **Example:** a lab technician accidentally drops a test tube, so that one reading is lost. Any row is equally likely to be missing.

Definition: MAR — Missing At Random

Whether a value is missing depends on *other columns we can see*, but not on the missing value itself. **Example:** men are less likely to answer a question about feelings. The missingness depends on *gender* (a column we have), not on the feeling value itself.

Definition: MNAR — Missing Not At Random

Whether a value is missing depends on the *missing value itself*. **Example:** people with very high income often refuse to report their income. The missingness depends on the income value we are trying to collect.

Why this matters

If data is **MCAR**, simply dropping or filling values is usually safe. If data is **MAR** or **MNAR**, careless filling can *bias* your results — the missing rows are special, not random. MNAR is the hardest case and often needs domain knowledge.

2.3 Detecting Missing Values

Before fixing anything, we count how many values are missing in each column.

```
import pandas as pd

# Count missing values per column
df.isnull().sum()

# Percentage missing per column (often more useful)
(df.isnull().mean() * 100).round(1)

```

Rule of thumb on how much is missing

- $<$**5%** missing: usually easy to fill.
- **5–30%** missing: fill carefully, think about the cause.
- $>$**50%** missing: consider dropping the whole column — there may be too little signal.

(These are guidelines, not strict rules.)

2.4 Strategies to Handle Missing Values

There are two big families: **delete** the missing data, or **impute** (fill in) the missing data.

2.4.1 Family A — Deletion

- **Listwise deletion (drop rows):** remove any row that has a missing value. Simple, but you lose data. Best when very few rows are affected and data is MCAR.
- **Drop a column:** if a column is mostly empty, remove the whole column.

```
df.dropna()                          # drop rows with ANY missing value
df.dropna(subset=['Salary'])         # drop rows missing Salary only
df.drop(columns=['SomeColumn'])      # drop an entire column

```

2.4.2 Family B — Imputation (filling in)

Imputation means replacing a missing value with a sensible estimate.

Definition: Imputation

The process of *filling in* missing values with substitute values (such as the mean, median, or most frequent value) so the dataset becomes complete.

Common imputation methods:

- **Mean:** replace with the column average. Good for numbers that are roughly symmetric.
- **Median:** replace with the middle value. Better when the column is skewed or has outliers.
- **Mode (most frequent):** replace with the most common value. Used for *categorical* columns.
- **Constant:** fill with a fixed value such as `0` or the word `"Unknown"`.
- **Forward / Backward fill:** copy the previous (or next) value. Used for *time-series* data.
- **KNN imputation:** look at the k most similar rows and average their values.
- **Iterative / MICE:** predict each missing column using the other columns with a model.

```
# Simple fills
df['Age']    = df['Age'].fillna(df['Age'].mean())       # mean
df['Age']    = df['Age'].fillna(df['Age'].median())     # median (robust)
df['Gender'] = df['Gender'].fillna(df['Gender'].mode()[0])  # mode (category)
df['City']   = df['City'].fillna('Unknown')             # constant

# Time-series fills
df['Temp'] = df['Temp'].ffill()   # forward fill
df['Temp'] = df['Temp'].bfill()   # backward fill

# scikit-learn imputers (work on whole arrays)
from sklearn.impute import SimpleImputer, KNNImputer
imp = SimpleImputer(strategy='median')
df[['Age','Salary']] = imp.fit_transform(df[['Age','Salary']])

```

Mean vs Median

The **mean** is dragged toward extreme values. If your column has an outlier (say a salary of 9,999,999), the mean becomes huge and unrealistic. The **median** ignores how far the extreme value is, so it is much safer when outliers are present. *When in doubt, use the median for numbers.*

2.5 Which Strategy Should I Pick?

Situation	Recommended approach
Very few rows missing (<5%), MCAR	Drop the rows (`dropna`).
Numeric column, symmetric, no outliers	Fill with the **mean**.
Numeric column, skewed or has outliers	Fill with the **median**.
Categorical (text) column	Fill with the **mode** or `"Unknown"`.
Time-series data	**Forward/backward fill** or interpolation.
Columns are strongly related	**KNN** or **Iterative (MICE)** imputation.
Column is mostly empty (>50%)	Consider **dropping the column**.

Key Takeaways

- Missing values are empty cells, shown as `NaN` in pandas.
- Causes: human error, sensor failure, non-response, merging, etc. Mechanisms: **MCAR**, **MAR**, **MNAR**.
- Two fixes: **delete** rows/columns, or **impute** (mean / median / mode / advanced).
- Use **median** for skewed numeric data, **mode** for categories.

3. Part 2 — Detecting & Removing Outliers

3.1 What Is an Outlier?

Definition: Outlier

A data point that is *very different* from the other observations — it lies unusually far above or below most of the data. Example: in a class of teenagers aged 13–18, a recorded age of **150** is an outlier.

3.2 Why Do Outliers Appear? (Causes)

- **Data entry errors:** typing 150 instead of 15, or adding an extra zero (650000 vs 65000).
- **Measurement errors:** a faulty sensor records a wild value.
- **Sampling problems:** accidentally including data from the wrong group.
- **Genuine extreme values:** sometimes the value is real and important (e.g. a billionaire's net worth). Not every outlier is a mistake!

3.3 Why Outliers Are a Problem

- They pull the **mean** and **standard deviation** away from typical values.
- They can confuse models like *linear regression* that try to fit every point.

- They make charts hard to read (one giant bar squashes everything else).

3.4 How to Detect Outliers

- **Box plot (visual):** points drawn beyond the "whiskers" are flagged as outliers.
- **Z-score (statistical):** how many standard deviations a point is from the mean. A common cut-off is $|z| > 3$. Works best when data is bell-shaped (normal).
- **IQR method (statistical):** based on quartiles; does *not* assume a bell shape. This is our main method.

3.5 The IQR Method — Step by Step

IQR stands for **Inter-Quartile Range**. It looks at the "middle 50%" of the data and flags anything that falls too far outside it.

Definition: Quartiles and IQR

Sort the data. **Q1** (first quartile) is the value below which 25% of the data falls. **Q3** (third quartile) is the value below which 75% falls. The **Inter-Quartile Range** is the distance between them:

$$\text{IQR} = Q3 - Q1$$

We then build a "fence." Anything outside the fence is an outlier:

$$\boxed{\text{Lower bound} = Q1 - 1.5 \times \text{IQR} \qquad \text{Upper bound} = Q3 + 1.5 \times \text{IQR}}$$

A point is an **outlier** if $x <$ Lower bound or $x >$ Upper bound.

Why 1.5?

The multiplier **1.5** is a well-known convention (introduced by statistician John Tukey). It captures almost all "normal" values while flagging genuine extremes. Use **3.0** instead of 1.5 if you only want to catch the *very* extreme outliers.

A tiny worked example by hand

Data: 5, 7, 8, 9, 10, 11, 12, 13, 14, 100

Suppose we compute $Q1 = 8$ and $Q3 = 13$. Then:

- $\text{IQR} = 13 - 8 = 5$
- Lower bound $= 8 - 1.5 \times 5 = 8 - 7.5 = 0.5$
- Upper bound $= 13 + 1.5 \times 5 = 13 + 7.5 = 20.5$

The value **100** is greater than 20.5, so **100 is an outlier**. Every other value lies inside [0.5, 20.5].

The same logic in Python:

```
Q1 = df['Salary'].quantile(0.25)
Q3 = df['Salary'].quantile(0.75)
IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

# Rows that ARE outliers
```

```
        outliers = df[(df['Salary'] < lower) | (df['Salary'] > upper)]

        # Keep only the rows that are NOT outliers
        df_clean = df[(df['Salary'] >= lower) & (df['Salary'] <= upper)]

```

3.6 What to Do With Outliers

You do not always delete them. Choose based on the situation:

- **Remove** the rows — when the value is clearly an error and you have plenty of data.
- **Cap / Winsorize** — replace the outlier with the boundary value (e.g. set anything above the upper bound *to* the upper bound). Keeps the row but tames the extreme.
- **Transform** — apply a log transform to squeeze large values closer together.
- **Keep** — when the outlier is a genuine, meaningful observation.

```
        # Capping (winsorizing) instead of removing
        Q1 = df["Salary"].quantile(0.25)
        Q3 = df["Salary"].quantile(0.75)

        IQR = Q3 - Q1

        lower = Q1 - 1.5 * IQR
        upper = Q3 + 1.5 * IQR

        df["Salary"] = df["Salary"].clip(lower, upper)

```

If value $<$ lower $\rightarrow$ lower

If value $>$ upper $\rightarrow$ upper

Otherwise $\rightarrow$ keep original value

Key Takeaways

- An outlier is a value far from the rest; it may be an error or a genuine extreme.
- The **IQR method**: $\text{IQR} = Q3 - Q1$; fences at $Q1 - 1.5\,\text{IQR}$ and $Q3 + 1.5\,\text{IQR}$.
- Options: remove, cap (clip), transform, or keep. Think before deleting!

4. Part 3 — Encoding Categorical Variables

4.1 Why Encoding?

Most machine-learning models do maths on numbers. They cannot directly understand text like "IT" or "Master." **Encoding** converts these text categories into numbers the model can use.

Definition: Encoding

Transforming *categorical* (text) data into a *numeric* form so it can be used by a machine-learning model.

4.2 First, Know Your Category Type: Nominal vs Ordinal

The right encoding depends on whether the categories have an order.

Definition: Nominal vs Ordinal

Nominal: categories with *no* natural order. Examples: City (Mumbai, Delhi), Department (IT, HR), Colour (Red, Blue).

Ordinal: categories that *do* have an order. Examples: Education (High School < Bachelor < Master < PhD), Size (Small < Medium < Large).

4.3 Label Encoding

Label encoding gives each category a whole number: $0, 1, 2, 3, \ldots$

Label encoding "Education"

$$\text{High School} \to 0, \quad \text{Bachelor} \to 1, \quad \text{Master} \to 2, \quad \text{PhD} \to 3$$

This is perfect here, because the numbers respect the real order (a PhD is "higher" than a Bachelor).

```
# Method 1: manual map (best for ORDINAL, you control the order)
order = {'High School': 0, 'Bachelor': 1, 'Master': 2, 'PhD': 3}
df['Education_Label'] = df['Education'].map(order)

# Method 2: pandas category codes (order is alphabetical unless specified)
df['Dept_Label'] = df['Department'].astype('category').cat.codes

# Method 3: scikit-learn (mainly for the TARGET column y)
from sklearn.preprocessing import LabelEncoder
df['Gender_Label'] = LabelEncoder().fit_transform(df['Gender'])

```

The hidden danger of label encoding

If you label-encode a **nominal** column, the model wrongly believes there is an order. Suppose City: Mumbai= 0, Delhi= 1, Pune= 2. A linear model may think "Pune > Delhi > Mumbai" and even that "Mumbai + Pune = Delhi ×2," which is nonsense. **Use label encoding only for ordinal data (or with tree-based models).** For nominal data, use one-hot encoding.

4.4 One-Hot Encoding

One-hot encoding creates a new column for *each* category and marks it with `1` (yes) or `0` (no).

One-hot encoding "Department"

Original	Dept_Finance	Dept_HR	Dept_IT	Dept_Sales
IT	0	0	**1**	0
HR	0	**1**	0	0
Sales	0	0	0	**1**
Finance	**1**	0	0	0

Each row has exactly one `1`. No fake ordering is created — all departments are treated as equal and separate.

```
        # Method 1: pandas (easiest)
        df = pd.get_dummies(df, columns=['Department', 'City'], drop_first=False)

        # Method 2: scikit-learn
        from sklearn.preprocessing import OneHotEncoder
        ohe = OneHotEncoder(sparse_output=False, handle_unknown='ignore')
        # 1. Encode
        encoded = ohe.fit_transform(df[['Department']])

        # 2. Convert to DataFrame
        encoded_df = pd.DataFrame(
        encoded,
        columns=ohe.get_feature_names_out(['Department'])
        )

        # 3. Join back
        df_final = pd.concat(
        [df1.drop('Department', axis=1), encoded_df],
        axis=1
        )

```

5. One-Hot Encoding in Scikit-Learn

```
ohe = OneHotEncoder(
sparse_output=False,
handle_unknown='ignore'
)
```

This creates a `OneHotEncoder` object with two settings.

5.1 1. OneHotEncoder()

A machine learning model cannot understand text categories directly.

For example:

Department
Sales
IT
HR

One-Hot Encoding converts this into:

Department_HR	Department_IT	Department_Sales
0	0	1
0	1	0
1	0	0

5.2 2. sparse_output=False

By default, One-Hot Encoding returns a **sparse matrix**, which stores only non-zero values to save memory.

```
encoded = ohe.fit_transform(df[['Department']])
```

Output:

```
<Compressed Sparse Row sparse matrix of shape (5,3)>
```

This format is memory-efficient but difficult to read.
With:

```
ohe = OneHotEncoder(sparse_output=False)
```

the output becomes a regular NumPy array:

```
array([
[0., 0., 1.],
[0., 1., 0.],
[1., 0., 0.]
])
```

Thus, `sparse_output=False` means:

> "Return the complete matrix instead of a compressed sparse representation."

5.3 3. handle_unknown='ignore'

This parameter handles categories that were not present during training.
Training data:

```
Sales
IT
HR
```

Suppose the encoder later encounters:

```
Finance
```

Without:

```
OneHotEncoder()
```

an error occurs:

```
ValueError: Found unknown categories ['Finance']
```

With:

```
OneHotEncoder(handle_unknown='ignore')
```

no error is raised. The unseen category is encoded as:

HR	IT	Sales
0	0	0

Thus, `handle_unknown='ignore'` means:

> "If a new category appears during prediction, do not raise an error; encode it as all zeros."

5.4 Summary

Parameter	Meaning
`OneHotEncoder()`	Converts categorical values into binary (0/1) columns.
`sparse_output=False`	Returns a normal NumPy array instead of a sparse matrix.
`handle_unknown='ignore'`	Prevents errors when unseen categories appear.

Two things to watch with one-hot encoding

1. **Too many columns (high cardinality):** a column with 1000 unique values becomes 1000 new columns. This slows things down and can hurt performance.
2. **The dummy-variable trap:** if you have k categories and keep all k columns, they are perfectly related (they always add up to 1). For *linear* models, drop one column using `drop_first=True` to avoid this redundancy.

5.5 Label vs One-Hot: Which One?

	Label Encoding	One-Hot Encoding
Best for	Ordinal data (has order)	Nominal data (no order)
Creates	One numeric column	One column per category
Adds order?	Yes (can be wrong)	No
Columns added	Few (compact)	Many (can explode)
Good with	Tree models, ordinal features	Linear models, nominal features

5.6 A Quick Note on High-Cardinality Columns

When a nominal column has *very many* categories (e.g. "Product ID" with thousands of values), one-hot creates too many columns. In that case people often use **frequency encoding** (replace category with how often it appears) or **target encoding** (replace with the average target value for that category). These are more advanced and beyond this module, but it is good to know they exist.

Key Takeaways

- Models need numbers, so we **encode** text categories.
- **Ordinal** (ordered) → **Label encoding**. **Nominal** (unordered) → **One-hot encoding**.
- Watch out for fake ordering (label) and column explosion / dummy trap (one-hot).

6. Part 4 — Full Practical Walk-through

Now we put everything together on one messy dataset, in the recommended order: **inspect** → **fix missing** → **remove outliers** → **encode**.

6.1 The Messy Dataset

We use a small employee dataset (20 rows). It was designed to contain every problem we just studied:

Column	Type	Problem hidden in it
EmployeeID	Numeric (ID)	None — just an identifier.
Age	Numeric	2 missing values; one impossible value (150).
Gender	Nominal	1 missing value; needs encoding.
Department	Nominal	1 missing value; needs one-hot encoding.
Education	Ordinal	Needs label encoding (it has an order).
Experience	Numeric	3 missing values.
Salary	Numeric	2 missing values; one huge outlier (9,999,999).
City	Nominal	Needs one-hot encoding.

The raw data looks like this when printed:

```
   EmployeeID   Age Gender Department   Education  Experience    Salary      City
1  25.0   Male      Sales    Bachelor         2.0   45000.0    Mumbai
2  30.0 Female         IT      Master         5.0   60000.0     Delhi
3   NaN   Male         HR    Bachelor         3.0   50000.0    Mumbai
4  35.0 Female    Finance         PhD        10.0   90000.0 Bangalore
5  28.0   Male         IT    Bachelor         4.0       NaN     Delhi
6 150.0 Female      Sales High School         1.0   35000.0      Pune
7  40.0    NaN         HR      Master        12.0   75000.0    Mumbai
8  33.0   Male    Finance    Bachelor         8.0   65000.0 Bangalore
9  29.0 Female         IT      Master         5.0   58000.0     Delhi
10  45.0   Male      Sales High School         NaN   48000.0      Pune
11  38.0 Female         NaN         PhD        15.0   95000.0    Mumbai
12  26.0   Male         IT    Bachelor         3.0 9999999.0     Delhi
13  50.0 Female    Finance      Master        20.0  110000.0 Bangalore
14  31.0   Male         HR    Bachelor         NaN       NaN      Pune
15  27.0 Female      Sales High School         2.0   38000.0    Mumbai
16   NaN   Male         IT      Master         7.0   70000.0     Delhi
17  42.0 Female    Finance         PhD        18.0  105000.0 Bangalore
18  36.0   Male      Sales    Bachelor         9.0   62000.0      Pune
19  34.0 Female         HR      Master        11.0   72000.0    Mumbai
20  39.0   Male         IT    Bachelor         NaN   68000.0     Delhi
```

6.2 Step 0 — Setup and Create the Data

```
import pandas as pd
import numpy as np

data = {
    'EmployeeID': list(range(1, 21)),
    'Age':        [25,30,np.nan,35,28,150,40,33,29,45,38,26,50,31,27,np.nan,42,36,34,39],
    'Gender':     ['Male','Female','Male','Female','Male','Female',np.nan,'Male','Female'
,'Male',
    'Female','Male','Female','Male','Female','Male','Female','Male','Female','Male'],
    'Department': ['Sales','IT','HR','Finance','IT','Sales','HR','Finance','IT','Sales',
    np.nan,'IT','Finance','HR','Sales','IT','Finance','Sales','HR','IT'],
    'Education':  ['Bachelor','Master','Bachelor','PhD','Bachelor','High School','Master'
,'Bachelor',
    'Master','High School','PhD','Bachelor','Master','Bachelor','High School','Master',
    'PhD','Bachelor','Master','Bachelor'],
    'Experience': [2,5,3,10,4,1,12,8,5,np.nan,15,3,20,np.nan,2,7,18,9,11,np.nan],
```

```
            'Salary':     [45000,60000,50000,90000,np.nan,35000,75000,65000,58000,48000,
            95000,9999999,110000,np.nan,38000,70000,105000,62000,72000,68000],
            'City':       ['Mumbai','Delhi','Mumbai','Bangalore','Delhi','Pune','Mumbai','
    Bangalore','Delhi',
            'Pune','Mumbai','Delhi','Bangalore','Pune','Mumbai','Delhi','Bangalore','Pune',
            'Mumbai','Delhi'],
        }
        df = pd.DataFrame(data)

```

6.3 Step 1 — Inspect and Find Missing Values

```
        print(df.isnull().sum())                # count missing per column
        print((df.isnull().mean()*100).round(1)) # percentage missing

```

Output:

```
EmployeeID    0        EmployeeID     0.0
Age           2        Age           10.0
Gender        1        Gender         5.0
Department    1        Department     5.0
Education     0        Education      0.0
Experience    3        Experience    15.0
Salary        2        Salary        10.0
City          0        City           0.0
```

So 9 values are missing in total. Experience is the worst column (15% missing), but all are well under 50%, so we will fill them rather than drop anything.

6.4 Step 2 — Impute Missing Values

We use the **median** for numeric columns (because Age and Salary contain outliers, and the median is robust), and the **mode** for categorical columns.

```
        df['Age']        = df['Age'].fillna(df['Age'].median())
        df['Experience'] = df['Experience'].fillna(df['Experience'].median())
        df['Salary']     = df['Salary'].fillna(df['Salary'].median())
        df['Gender']     = df['Gender'].fillna(df['Gender'].mode()[0])
        df['Department'] = df['Department'].fillna(df['Department'].mode()[0])

        print(df.isnull().sum().sum())   # should be 0

```

The values used to fill:

Column	Method	Fill value
Age	median	34.5
Experience	median	7.0
Salary	median	66,500.0
Gender	mode	Male
Department	mode	IT

After this step, `df.isnull().sum().sum()` returns **0** — no missing values remain.

Notice the power of the median

The real salaries sit around 35k–110k, but row 12 has 9,999,999. If we had used the *mean* to fill Salary, the fill value would have been pulled up to roughly 560,000 — a nonsense salary! The median (66,500) stays realistic.

6.5 Step 3 — Detect and Remove Outliers (IQR)

We apply the IQR method to the two numeric columns that can have extreme values: **Age** and **Salary**.

```
def iqr_outliers(series):
Q1, Q3 = series.quantile(0.25), series.quantile(0.75)
IQR = Q3 - Q1
lower, upper = Q1 - 1.5*IQR, Q3 + 1.5*IQR
return lower, upper

for col in ['Age', 'Salary']:
lo, hi = iqr_outliers(df[col])
print(col, '-> keep between', lo, 'and', hi)

# Keep only rows inside the bounds for BOTH columns
mask = pd.Series(True, index=df.index)
for col in ['Age', 'Salary']:
lo, hi = iqr_outliers(df[col])
mask &= df[col].between(lo, hi)

df = df[mask].reset_index(drop=True)

```

Computed bounds and flagged outliers:

Column	Q1	Q3	IQR	Bounds (lower, upper)	Outlier row
Age	29.75	39.25	9.5	(15.5, 53.5)	ID 6 (Age = 150)
Salary	56000	78750	22750	(21875, 112875)	ID 12 (Salary = 9,999,999)

Two rows are removed (the age of 150 and the salary of 9,999,999). The dataset goes from **20 rows to 18 rows**.

6.6 Step 4 — Encode the Categorical Columns

We treat each categorical column according to its type:

- **Education** (ordinal) → label encoding with the correct order.
- **Gender** (nominal, only two values) → simple 0/1 label encoding.
- **Department** and **City** (nominal, many values) → one-hot encoding.

```
# Education: ORDINAL -> label encode with a defined order
edu_order = {'High School': 0, 'Bachelor': 1, 'Master': 2, 'PhD': 3}
df['Education_Label'] = df['Education'].map(edu_order)

# Gender: only two values -> 0/1
df['Gender_Label'] = df['Gender'].map({'Male': 0, 'Female': 1})

# Department & City: NOMINAL -> one-hot encode
df = pd.get_dummies(df, columns=['Department', 'City'],
prefix=['Dept', 'City'])

```

```
        # (optional) turn True/False dummies into 1/0 integers
        bool_cols = df.select_dtypes(include='bool').columns
        df[bool_cols] = df[bool_cols].astype(int)

```

6.7 Step 5 — The Clean, Model-Ready Dataset

After all four steps the data is complete (no blanks), free of the extreme outliers, and fully numeric. The final dataset has **18 rows and 16 columns**. A slice of it:

```
   EmployeeID  Age  Gender_Label  Education_Label  Experience   Salary  Dept_Finance  Dept_HR
Dept_IT  Dept_Sales  City_Bangalore  City_Delhi  City_Mumbai  City_Pune
1 25.0             0               1         2.0  45000.0             0        0        0
1               0           0            1          0
2 30.0             1               2         5.0  60000.0             0        0        1
0               0           1            0          0
3 34.5             0               1         3.0  50000.0             0        1        0
0               0           0            1          0
4 35.0             1               3        10.0  90000.0             1        0        0
0               1           0            0          0
5 28.0             0               1         4.0  66500.0             0        0        1
0               0           1            0          0
7 40.0             0               2        12.0  75000.0             0        1        0
0               0           0            1          0
```

(Notice row 6 is gone — it was the age-150 outlier — and row 5's Salary is now the median 66,500 that we imputed.)

6.8 Putting It All Together (one clean script)

```
        import pandas as pd
        import numpy as np

        # 1. LOAD ---------------------------------------------------------------
        df = pd.read_csv('messy_employees.csv')   # or build as in Step 0

        # 2. IMPUTE MISSING -----------------------------------------------------
        for col in ['Age', 'Experience', 'Salary']:
        df[col] = df[col].fillna(df[col].median())
        for col in ['Gender', 'Department']:
        df[col] = df[col].fillna(df[col].mode()[0])

        # 3. REMOVE OUTLIERS (IQR) ----------------------------------------------
        mask = pd.Series(True, index=df.index)
        for col in ['Age', 'Salary']:
        Q1, Q3 = df[col].quantile(0.25), df[col].quantile(0.75)
        IQR = Q3 - Q1
        mask &= df[col].between(Q1 - 1.5*IQR, Q3 + 1.5*IQR)
        df = df[mask].reset_index(drop=True)

        # 4. ENCODE -------------------------------------------------------------
        df['Education_Label'] = df['Education'].map(
        {'High School': 0, 'Bachelor': 1, 'Master': 2, 'PhD': 3})
        df['Gender_Label'] = df['Gender'].map({'Male': 0, 'Female': 1})
        df = pd.get_dummies(df, columns=['Department', 'City'],
        prefix=['Dept', 'City'])

        print(df.shape)           # (18, 16)
        print(df.isnull().sum().sum())  # 0

```

7. Summary & Cheat Sheet

Key formulas

$$\text{IQR} = Q3 - Q1, \qquad \text{Lower} = Q1 - 1.5\,\text{IQR}, \qquad \text{Upper} = Q3 + 1.5\,\text{IQR}$$

Decision summary

Task	**Quick rule**
Missing numeric (clean)	Fill with **mean**.
Missing numeric (skewed/outliers)	Fill with **median**.
Missing categorical	Fill with **mode** or `"Unknown"`.
Outliers	Detect with **IQR**; then remove, cap, transform, or keep.
Ordinal category	**Label encode** (define the order yourself).
Nominal category	**One-hot encode** (`pd.get_dummies`).